

Group: Nguyen Minh Khanh & Truong Minh Son

2.2. Secant method

a. Define secant in python:

Using this form of Secant:

$$x^{(k+1)} = x^{(k)} - g(x^{(k)}) \frac{x^{(k)} - x^{(k+1)}}{g(x^{(k)}) - g(x^{(k+1)})}$$

We define it in python:

```
denominator = function(x_current) - function(x_previous)
x_next = x_current - function(x_current) * (x_current - x_previous) / denominator
```

and stop the iteration after reaching ε

```
if abs(x_next - x_current) < abs(x_current) * epsilon:
    return x_next, iteration
```

b. Define function $g(x)$ and solution:

```
def g(x: float) -> float:
    return (2 * x - 1) ** 2 + 4 * (4 - 1024 * x) ** 4
```

The result is:

```
Secant method for 2.2
Group: Nguyen Minh Khanh & Truong Minh Son
root = 0.0038664094745
g(root) = 0.984605239109
iterations = 18
```

FULL CODE

```
"""Secant method for 2.2"""
"""Group: Nguyen Minh Khanh & Truong Minh Son"""

from collections.abc import Callable

def secant(
    function: Callable[[float], float],
    x_previous: float,
    x_current: float,
    epsilon: float,
) -> tuple[float, int]:
    iteration = 0

    while True:
        denominator = function(x_current) - function(x_previous)
        if denominator == 0:
            raise ZeroDivisionError("The secant slope is zero.")

        x_next = x_current - function(x_current) * (x_current - x_previous) /
denominator
        iteration += 1

        if abs(x_next - x_current) < abs(x_current) * epsilon:
            return x_next, iteration

        x_previous, x_current = x_current, x_next

if __name__ == "__main__":
    def g(x: float) -> float:
        return (2 * x - 1) ** 2 + 4 * (4 - 1024 * x) ** 4

    root, iterations = secant(
        function=g,
        x_previous=0.0,
        x_current=1.0,
        epsilon=1e-5,
    )
    print( "Secant method for 2.2\n"\
        "Group: Nguyen Minh Khanh & Truong Minh Son"
    )
    print(f"    root = {root:.12g}")
    print(f"    g(root) = {g(root):.12g}")
    print(f"    iterations = {iterations}")
```